

host_agent

Which houses in Exeter have a basement ?

planning_agent

User wants to know which houses in Exeter have a basement. Provide the general steps to solve this geospatial query.

The area is Exeter, so there will be 1 entry for Exeter. The entity to search for is houses, specifically those that have a basement. There may be multiple entries for houses in Exeter, and the condition is that they must have a basement.

output steps: ["Find Exeter 1 area", "Search for houses in Exeter as many search results", "Apply condition for houses that have a basement"]

named_area

Find the area polygon for Exeter.

Tool call_os_ngd with args : {'filters': None, 'bbox': None, 'polygon_or_point': True, 'street_address': None, 'filename': 'exeter_area_polygon'}

Tool call_os_ngd returned result : ['Search results have been found. \n Artifact generated is : exeter_area_polygon,\n Description is : A geopandas dataframe containing named area data with no filters applied for the query (so remember no search is performed here so spurious entities present) Find the area polygon for Exeter..,\n count of records fetched : 98.\n ',]

Filter the artifact 'exeter_area_polygon' to find the specific polygon for Exeter. Save the filtered results as 'filtered_exeter_area_polygon'.

Tool generate_metadata_for_artifacts with args : {'artifact_names': ['exeter_area_polygon']}

Tool generate_metadata_for_artifacts returned result : ...

Tool Code Executor

```
def filter_exeter_area_polygon(data: list):
    gdf = data[0]
    # Filter for polygons specifically representing Exeter
    # Use multiple columns for accuracy: name1_text, description, descriptiongroup
    filtered = gdf[((gdf['name1_text'].str.lower() == 'exeter') |
```

```

        (gdf['name1_text'].str.lower().str.contains('exeter')) |
        (gdf['description'].str.lower() == 'city') |
        (gdf['description'].str.lower().str.contains('exeter')) |
        (gdf['descriptiongroup'].str.lower() == 'settlement'))]
# Further filter to avoid partial matches if possible
filtered = filtered[(filtered['name1_text'].str.lower() == 'exeter') |
                    (filtered['description'].str.lower().str.contains('exeter')) |
                    (filtered['description'].str.lower() == 'city')]
summary = f"Filtered to {len(filtered)} polygon(s) for Exeter."
return [summary, 'filtered_exeter_area_polygon', 'GeoDataFrame containing only the polygon(s) representing Exeter city area.', filtered]

```

Filtered to 1 polygon representing Exeter. The result is saved as 'filtered_exeter_area_polygon', which contains only the polygon(s) for the Exeter city area. If you need further details or analysis on this filtered area, let me know!

The specific polygon for Exeter has been filtered and saved as 'filtered_exeter_area_polygon'. If you need further details or analysis on this area, feel free to ask!

buildings

Search for houses in the area defined by 'filtered_exeter_area_polygon' that have a basement.

Tool call_os_ngd with args : {'filters': None, 'bbox': 'filtered_exeter_area_polygon', 'polygon_or_point': True, 'street_address': None, 'filename': 'all_buildings_in_filtered_exeter_area_with_basement'}

Tool call_os_ngd returned result : ['Multiple search results have been found in multiple datasets. A summary of each is provided.\n Artifacts generated are : ['buildings_all_buildings_in_filtered_exeter_area_with_basement', 'buildingline_all_buildings_in_filtered_exeter_area_with_basement', 'buildingpart_all_buildings_in_filtered_exeter_area_with_basement'],\n Descriptions are : ["A geopandas dataframe containing building data with filters and bbox applied for the query Search for houses in the area defined by 'filtered_exeter_area_polygon' that have a basement. using filters None.", "A geopandas dataframe containing building line data with filters and bbox applied for the query Search for houses in the area defined by 'filtered_exeter_area_polygon' that have a basement. using filters None.", "A geopandas dataframe containing building part data with filters and bbox applied for the query Search for houses in the area defined by 'filtered_exeter_area_polygon' that have a basement. using filters None."],\n counts of records fetched are : [69169, 69342, 85480].\n ', [, ...]]

From the artifact 'buildings_all_buildings_in_filtered_exeter_area_with_basement', filter and return only those buildings that meet the definition of a house (buildinguse_addresscount_residential > 0 and buildinguse_addresscount_total = 1) and have a basement. Save the filtered results as

'houses_with_basement_in_filtered_exeter_area'.

Tool generate_metadata_for_artifacts with args : {'artifact_names': ['buildings_all_buildings_in_filtered_exeter_area_with_basement']}

Tool generate_metadata_for_artifacts returned result : ...

Tool Code Executor

```
def filter_houses_with_basement(data: list):
    import geopandas as gpd
    df = data[0]
    # Filter for houses: residential > 0 and total = 1
    house_mask = (df['buildinguse_addresscount_residential'] > 0) & (df['buildinguse_addresscount_total'] == 1)
    # Filter for basement presence: 'Present' in basementpresence
    basement_mask = df['basementpresence'] == 'Present'
    filtered = df[house_mask & basement_mask]
    summary = f"Filtered {len(filtered)} houses with a basement from {len(df)} total buildings."
    artifact_name = 'houses_with_basement_in_filtered_exeter_area'
    artifact_description = "GeoDataFrame of buildings in the filtered Exeter area that meet the definition of a house (single residential address, no commercial/other)"
    return [summary, artifact_name, artifact_description, filtered]
```

Filtered 480 houses with a basement from 69,169 total buildings in the filtered Exeter area. The results are saved as 'houses_with_basement_in_filtered_exeter_area', which contains a GeoDataFrame of buildings that meet the definition of a house (single residential address, no commercial/other) and have a basement present.

Search Results: - 480 houses with a basement were found in the area defined by 'filtered_exeter_area_polygon'. - These results use the definition of a house: buildinguse_addresscount_residential > 0 and buildinguse_addresscount_total = 1 (i.e., a single residential address, no commercial/other use). - The filtered results are saved in the artifact: houses_with_basement_in_filtered_exeter_area.

Artifact: - houses_with_basement_in_filtered_exeter_area: GeoDataFrame of buildings in the filtered Exeter area that meet the definition of a house and have a basement present.

plotting_agent

Plot the artifact 'houses_with_basement_in_filtered_exeter_area' to show the locations of all houses in Exeter that have a basement.

Tool generate_metadata_for_artifacts with args : {'artifact_names': ['houses_with_basement_in_filtered_exeter_area']}

Tool generate_metadata_for_artifacts returned result : ...

Tool Code Executor

```
def plot_houses_with_basement_exeter(data: list):
    import geopandas as gpd
    import folium
    from shapely.geometry import mapping

    # Get the geodataframe
    gdf = data[0].copy()

    # Ensure CRS is EPSG:4326 for folium
    if gdf.crs is not None and gdf.crs.to_string() != 'EPSG:4326':
        gdf = gdf.to_crs(epsg=4326)

    # Center the map on the mean location of all houses
    centroid = gdf.geometry.centroid.unary_union.centroid
    m = folium.Map(location=[centroid.y, centroid.x], zoom_start=13, tiles='cartodbpositron')

    # Add all houses as polygons
    folium.GeoJson(
        gdf,
        name='Houses with Basement',
        style_function=lambda x: {
            'fillColor': '#3186cc',
            'color': '#225577',
            'weight': 1,
            'fillOpacity': 0.5
        },
        tooltip=folium.GeoJsonTooltip(fields=['description', 'buildingage_period', 'numberoffloors', 'geometry_area_m2'])
    ).add_to(m)

    # Save map
    map_filename = 'houses_with_basement_exeter_map.html'
    m.save(map_filename)

    # Prepare summary
    summary = f"Plotted {len(gdf)} houses in Exeter with a basement. First 5 entries:\n" + \
        str(gdf[['description', 'buildingage_period', 'numberoffloors', 'geometry_area_m2']].head(5))

    return [
        summary,
        'houses_with_basement_exeter_map',
        'A folium map showing the locations and outlines of all houses in Exeter with a basement. Each house polygon is interactive with tooltips showing descriptions',
        map_filename
    ]

# The function expects a list with the geodataframe as the first element.
```

Error: Object of type Timestamp is not JSON serializable

Tool Code Executor

```
def plot_houses_with_basement_exeter(data: list):
    import geopandas as gpd
    import folium
    import pandas as pd

    # Get the geodataframe
    gdf = data[0].copy()

    # Convert all datetime columns to string to avoid serialization issues
    for col in gdf.columns:
        if pd.api.types.is_datetime64_any_dtype(gdf[col]):
            gdf[col] = gdf[col].astype(str)

    # Ensure CRS is EPSG:4326 for folium
    if gdf.crs is not None and gdf.crs.to_string() != 'EPSG:4326':
        gdf = gdf.to_crs(epsg=4326)

    # Center the map on the mean location of all houses
    centroid = gdf.geometry.centroid.unary_union.centroid
    m = folium.Map(location=[centroid.y, centroid.x], zoom_start=13, tiles='cartodbpositron')

    # Add all houses as polygons
    folium.GeoJson(
        gdf,
        name='Houses with Basement',
        style_function=lambda x: {
            'fillColor': '#3186cc',
            'color': '#225577',
            'weight': 1,
            'fillOpacity': 0.5
        },
        tooltip=folium.GeoJsonTooltip(fields=['description', 'buildingage_period', 'numberoffloors', 'geometry_area_m2'])
    ).add_to(m)

    # Save map
    map_filename = 'houses_with_basement_exeter_map.html'
    m.save(map_filename)

    # Prepare summary
    summary = f"Plotted {len(gdf)} houses in Exeter with a basement. First 5 entries:\n" + \
        str(gdf[['description', 'buildingage_period', 'numberoffloors', 'geometry_area_m2']].head(5))

    return [
        summary,
```

```

    'houses_with_basement_exeter_map',
    'A folium map showing the locations and outlines of all houses in Exeter with a basement. Each house polygon is interactive with tooltips showing description, age period, number of floors, and area.',
    map_filename
]

# The function expects a list with the geodataframe as the first element.
```

Plotted 480 houses in Exeter with a basement. First 5 entries: description buildingage_period numberoffloors \ 41954 Mid-Terrace House 2000-2009 2.0
42255 Mid-Terrace House 1990-1999 3.0
10856 Mid-Terrace House Pre-1919 2.0
48700 Semi-Detached House Pre-1919 2.0
2364 Mid-Terrace House Pre-1837 3.0

	geometry_area_m2
41954	79.296
42255	33.898
10856	30.490
48700	680.922
2364	50.795

A folium map showing the locations and outlines of all houses in Exeter with a basement. Each house polygon is interactive with tooltips showing description, age period, number of floors, and area.

Map file: houses_with_basement_exeter_map.html

host_agent

There are 480 houses in Exeter that have a basement. These houses have been identified as single residential buildings (no commercial or other use) with a basement present.

A map has been generated showing the locations and outlines of all these houses in Exeter. Each house polygon on the map is interactive, displaying details such as description, age period, number of floors, and area.

If you would like to view or download the map, let me know! If you need more details about specific houses or want to refine your search, please specify your requirements.